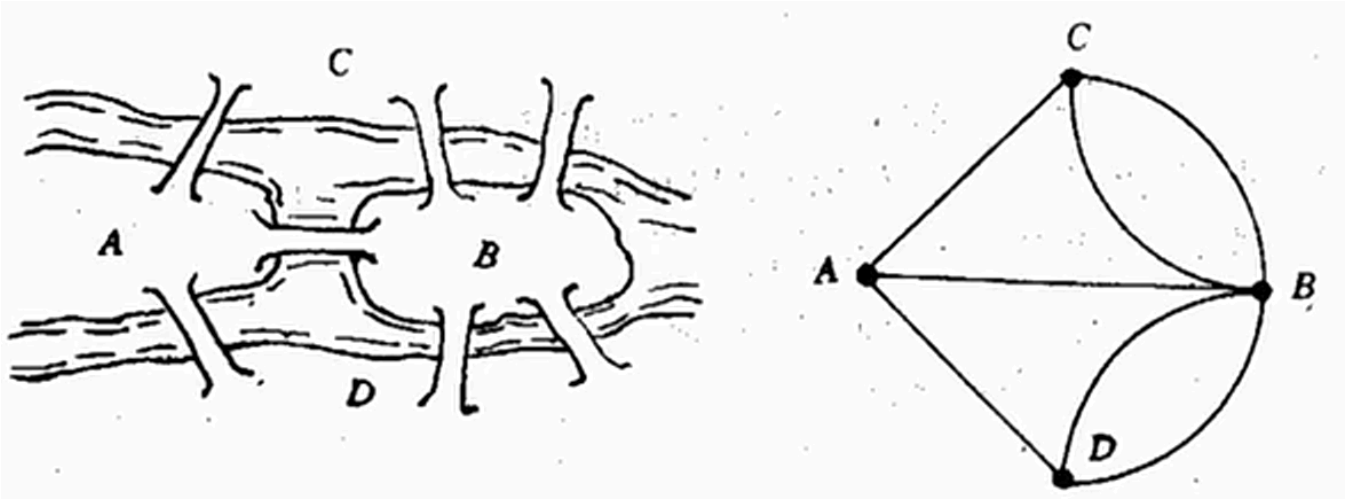
欧拉路

“一笔画”游戏就是欧拉路问题：给定一个图，要求一笔连续地画出整个图，每条边都必须经过且只能经过一次，点可以重复经过。

问题来自于中世纪数学家欧拉的七桥问题。把这条一笔画路线称为欧拉路，如果还要求起点和终点相同，称为欧拉回路。



欧拉路问题主要有两个：是否存在欧拉路、打印出欧拉路。

问题的解决通过处理度（Degree），一个点上连接的边的数量称为这个点的度数。在无向图中，如果度数是奇数，称这个点为奇点，否则称为偶点。在有向图中，有出度和入度。

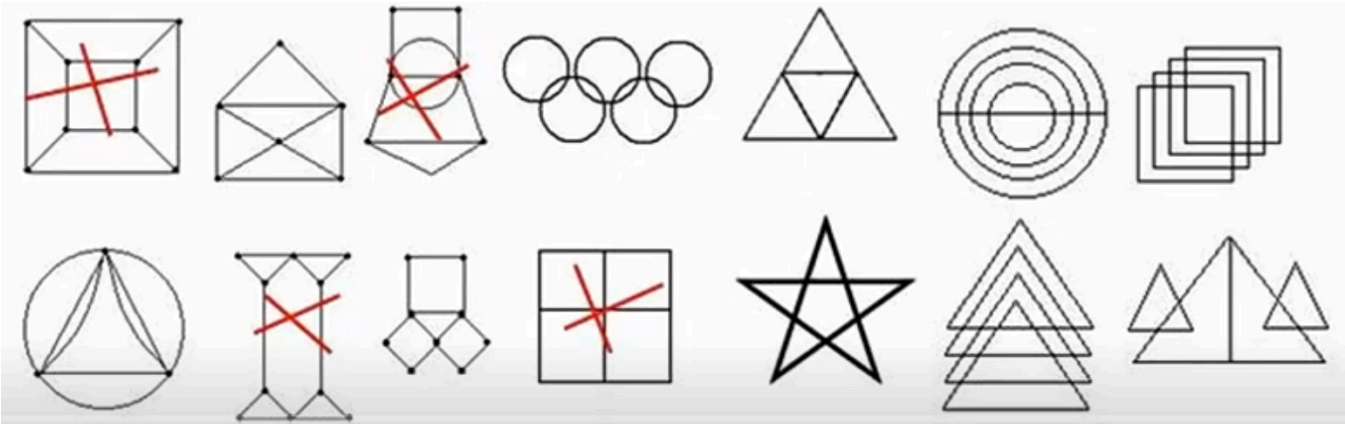
1.1 欧拉路和欧拉回路的存在性判断

首先，图一个是连通图，用DFS和并查集判断连通性。

其次，判断图是否存在欧拉路或欧拉回路。

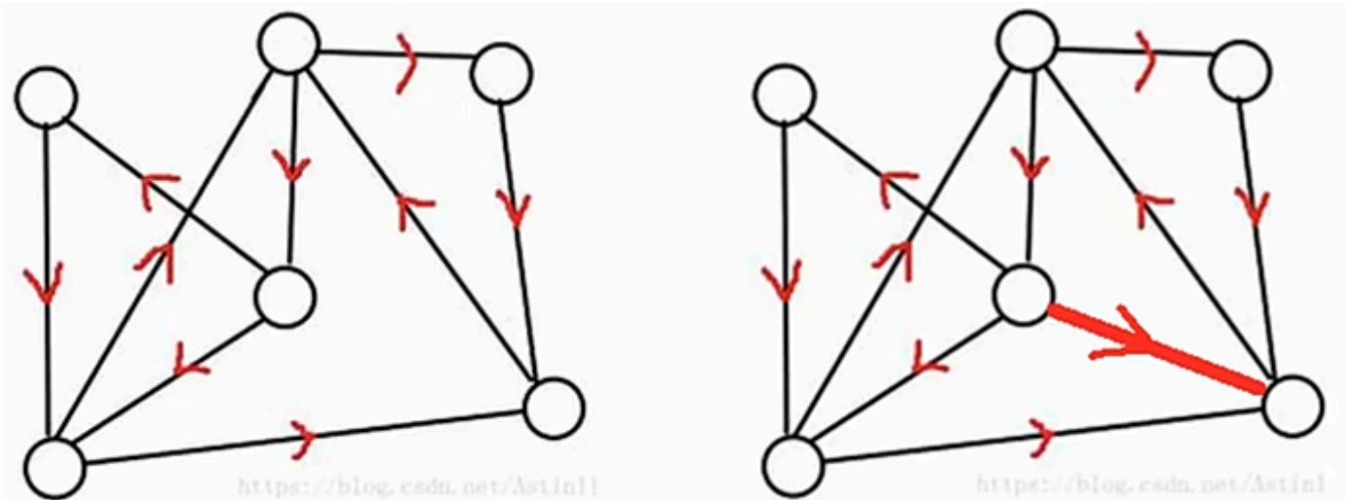
1.1.1 无向连通图的判断条件。

- 如果图中的点全都是偶点，则存在欧拉回路，任意点都可以作为起点和终点。
- 如果只有两个奇点，则存在欧拉路，其中一个奇点是起点，另一个是终点。
- 不可能出现有奇数个奇点的无向图，请同学们思考。



1.1.2 有向连通图的判断条件。

- 把一个点的出度记为1，入度记为-1，这个点所有出度和入度相加，就是它的度数。
- 一个有向图存在欧拉回路的条件是当且仅当该图所有点的度数为0.
- 如果只有一个度数为1的点，一个度数为-1的点，其它所有点的度数为0，那么存在欧拉路径，其中度数为1的是起点，度数为-1的是终点。



1.1.3 混合图的欧拉回路

有的图不是单纯的有向图或无向图，而是两者的混合，同时存在有向边和无向边。

这是一类比较困难的问题，需要用最大流求解。后面我们会继续深入探究。

1.1.4 例题 UVA10054 The Necklace

题目描述

有一种由彩色珠子连接而成的项链。每个珠子的两半由不同颜色组成。如图所示，相邻两个珠子在接触的地方颜色相同。现在有一些零碎的珠子，需要确认它们是否可以复原成完整的项链。

输入格式

- 第一行为测试数据组数 T 。
- 每组数据的第一行是一个整数 n $(5 \leq n \leq 1000)$，表示珠子的个数。接下来的 n 行每行包含两个整数，即珠子两半的颜色。颜色用 $1 \sim 50$ 的整数来表示。

输出格式

对于每组数据，输出测试数据编号和方案。

- 如果无解，输出 `some beads may be lost` 。
- 方案的格式和输入相同，共 n 行。
  每行两个整数描述一个珠子（从左到右），其中第一个整数表示左半的颜色，第二个整数表示右半的颜色。对于 $1 \leq i \leq n-1$,第 i 行的第二个数必须等于第 $i+1$ 行上的第一个数，且第 n 行的第二个数必须等于第一行的第一个数（因为项链是环形的）。
  如果有多解，输出任意一组即可。
- 在相邻两组输出之间应有一个空行。

输入输出样例 #1

输入 #1

```
2
5
1 2
2 3
3 4
4 5
5 6
5
2 1
2 2
3 4
3 1
2 4
```

输出 #1

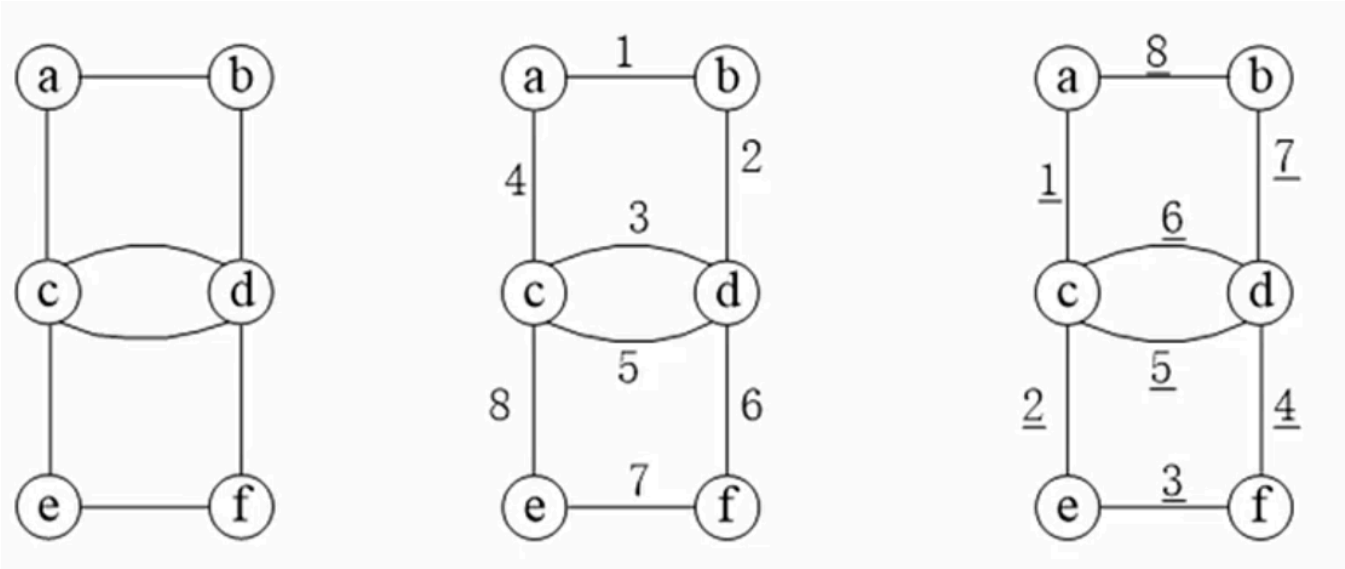
```
Case #1
some beads may be lost

Case #2
2 1
1 3
3 4
4 2
2 2
```

- 本题是典型的无向图求欧拉回路。
- 首先，判断所有的点是否为偶点，如果存在奇点，则没有欧拉回路;
- 其次，判断所给的图是否连通（用DFS或并查集实现），不连通也不是欧拉回路。不过本题比较简单，不用判断连通性。

1.2 输出一个欧拉回路

1.2.1 用DFS输出一个欧拉回路

对一个无向连通图做DFS，就输出了一个欧拉回路。



从a点开始DFS，DFS的对象是边，2图边上的数字是DFS访问的顺序。也可以有别的顺序。为了帮助理解，特意选了一个不太“好”的顺序。3图边的下划线数字是DFS回溯的顺序，它正好是一个欧拉回路。

代码输出的路径，实际上是从终点到起点的一条路径。对于无向图，因为起点和终点都是一个点，所以没有关系。

如果是有向图，输出的是一条逆序的路径，可以用栈把逆序按正序打印出来。

1.2.2 用栈模拟递归输出欧拉回路

上面用递归输出欧拉回路。不过递归常见的问题是爆栈，如果数据很大，就不能直接用递归，需要自己写栈模拟递归。

下面给出一道欧拉路的经典题。 当n=6时，建模的图有10^5个点，10^6条边。如果直接递归，可能爆栈。

1.2.3 例题 POJ 1780 CODE - Code

题目描述

KEY 公司开发出一种新的保险箱。要打开保险箱，不需要钥匙，但需要输入一个正确的、由 n 位数字组成的密码。当正确地输入最后一位密码后，保险箱就立刻打开了。保险箱上没有“确定”键。当你输入超过 n 位数字，则只有最后 n 位数字有效。例如，对一种 4 位数字编码的型号，如果正确的编码为 4567，你想输入的编码为 1234567890，则保险箱的门会在你输入数字 7 后马上就打开了。

为了达到这种效果所需要设计的软件其实很简单：

对 n 位数字编码的型号，保险箱始终处于 $10^{(n-1)}$ 种内部状态之一。

保险箱的当前状态只需用最后输入的 n−1 位数字表示，其中有一种状态（例如，对前面的例子，就是 456）被记为“开锁状态”。

如果保险箱处于“开锁状态”，且输入最后一位正确的数字（例如，在上面的例子中就是 7），保险箱的门就打开了；
否则保险箱切换到对应的新状态。

例如，如果保险箱的当前状态为 456，接着输入 8，则保险箱的状态切换到 568。

开保险箱的一个繁琐的策略是一位接一位地输入所有可能的编码。然而，在最坏情况下，这需要按键 $n\times10^n$ 次。而选择一个好的数字序列，最多只需要按键 10^n+n-1 次就可以打开保险箱。你需要做的就是找到一个数字序列包含所有的 n 位数一次且仅一次。

KEY 公司宣称，对军用型号（n=6），当今最快的计算机也需要数十亿年的时间才能找到这样的数字序列，但是很显然，他们不知道有些程序员能在几分钟就能找到这样的数字序列。

输入格式

输入文件中包含多个测试数据。每个测试数据为一个整数 n(1≤n≤6)。

输入文件最后一行为 0，表示输入结束。

输出格式

对输入文件中的每个测试数据，输出一行包含 位的数字序列，使得每个 n 位数出现一次且仅一次。

输入输出样例 #1

输入 #1

```
1
2
0
```

输出 #1

```
0123456789
00102030405060708091121314151617181922324252627282933435363738394454647484955657585966768697787988
```